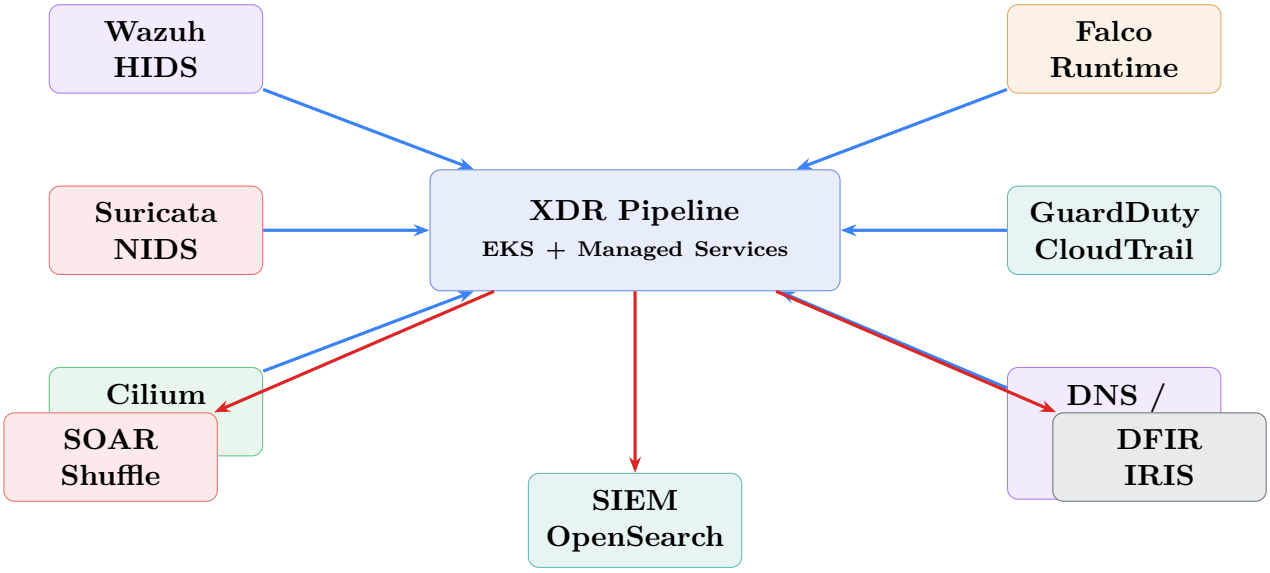
XDR Security Architecture

AWS Cloud Deployment

EKS · Managed Services · Multi-VPC



Fontys University of Applied Sciences

Group Project — Security Engineering

March 26, 2026

Contents

1 Overview — What Are We Building? 2

2 Network Layout — VPCs and Zones 2

3 Traffic Flow — From the Internet to Your Pod 3

4 Detection Layers — The Seven Sensors 4
- 4.1 Layer 1 — Tetragon (Linux Kernel, $< 1\,\mu s$) . . . 4
- 4.2 Layer 2 — Falco (Container Runtime, milliseconds) . . . 5
- 4.3 Layer 3 — Wazuh HIDS (Host, 50 ms — seconds) . . . 6
  - 4.3.1 Wazuh: Standalone vs. XDR Source . . . 6
- 4.4 Layer 4 — Cilium + Hubble (East-West, sub-second) . . . 8
- 4.5 Layer 5 — Suricata NIDS (Network, near real-time) . . . 9
- 4.6 Layer 6 — AWS-Native Security Services (GuardDuty, CloudTrail, etc.) . . . 9
- 4.7 Layer 7 — DNS Monitoring + Identity / Auth Logs . . . 10

5 The XDR Pipeline — Connecting All the Dots 10
- 5.1 The Message Queue — Why Amazon MSK Matters . . . 11
- 5.2 ML Tiers and the Fast Path . . . 12

6 SIEM, SOAR, and XDR — What Each Platform Does 12

7 SDN in AWS — Why There Is No Controller 13

8 The Tiered Defense Model 14

9 Attack Scenario Walkthroughs 15
- 9.1 Scenario 1 — DNS Tunnel (C2 over DNS) . . . 15
- 9.2 Scenario 2 — Lateral Movement (Zone B → Zone A) . . . 15
- 9.3 Scenario 3 — Brute Force + Credential Abuse . . . 16
- 9.4 Scenario 4 — DDoS Attack . . . 16
- 9.5 Scenario 5 — Container Compromise . . . 16

10 AWS Tool Reference 16

11 Gaps in the AWS Deployment 18

12 AWS Deployment Metrics Summary 20

Overview — What Are We Building?

The Shopping Mall

Think of your AWS environment as a large, modern shopping mall. The shops are your EC2 instances and EKS pods. The mall spans several wings with locked doors between them (VPCs and Security Groups). Customers are network connections arriving from the internet. This document describes the full security staff, CCTV system, and emergency response team for that mall — deployed entirely on AWS.

This document covers the XDR (Extended Detection and Response) architecture for the **AWS Cloud** deployment. The stack runs on **EKS** (Elastic Kubernetes Service) and integrates AWS-native managed services alongside open-source security tooling.

AWS Deployment at a Glance

- **Orchestration:** EKS (Kubernetes on EC2) for all security workloads
- **Network fabric:** VPC per zone, Transit Gateway for inter-zone routing
- **Managed services:** Amazon MSK (Kafka), Managed Flink, OpenSearch Service, Bedrock
- **Cloud-native security:** GuardDuty, Security Hub, CloudTrail, Shield Advanced, WAF
- **Monthly cost:** $15,000–17,000 (managed services trade OPEX for zero operational overhead)

Network Layout — VPCs and Zones

Wings of the Mall

Each VPC is a separate wing of the mall with its own locked entrance. The Transit Gateway is the central corridor that connects all wings — but the doors between wings are locked by default. To cross zones, you need an explicit rule.

The AWS deployment segments workloads into isolated VPCs connected via Transit Gateway.

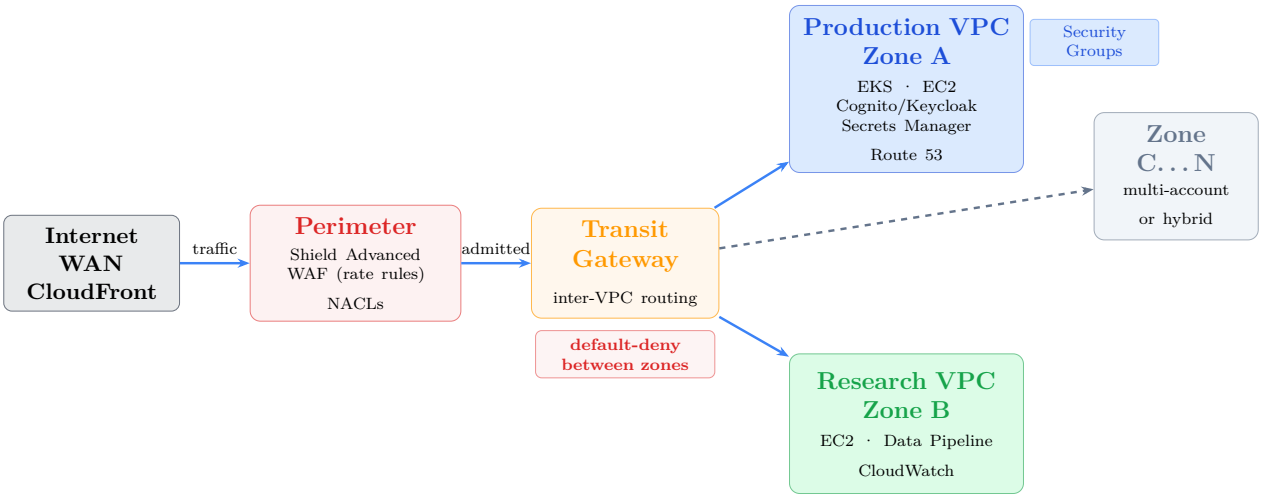


Figure 1: AWS network layout. Each zone is an isolated VPC. All inter-zone traffic routes through the Transit Gateway and is denied by default.

Key AWS Network Services

- **VPC (Virtual Private Cloud):** An isolated network segment. Production and Research workloads live in separate VPCs so a breach in one cannot directly reach the other.
- **Transit Gateway:** Acts as a central router connecting all VPCs. Routing tables define what is allowed to cross zones. Default: nothing.
- **Security Groups:** Stateful virtual firewalls attached to each EC2 or EKS node. The Enforcement API modifies these via `boto3` to block IPs in response to threats.
- **Network ACLs:** Stateless subnet-level packet filters — a second layer of defence at the subnet boundary.
- **AWS Shield Advanced:** Managed DDoS protection at the AWS edge. Absorbs volumetric attacks before they reach your VPCs.
- **AWS WAF:** Web Application Firewall on CloudFront / ALB. Blocks SQL injection, XSS, and custom rate rules.

Traffic Flow — From the Internet to Your Pod

The Journey of a Network Packet

Every connection arriving at your AWS environment passes through multiple checkpoints. At each checkpoint, a different sensor records what it sees. Even if an attacker slips through one checkpoint undetected, the next checkpoint will catch them.

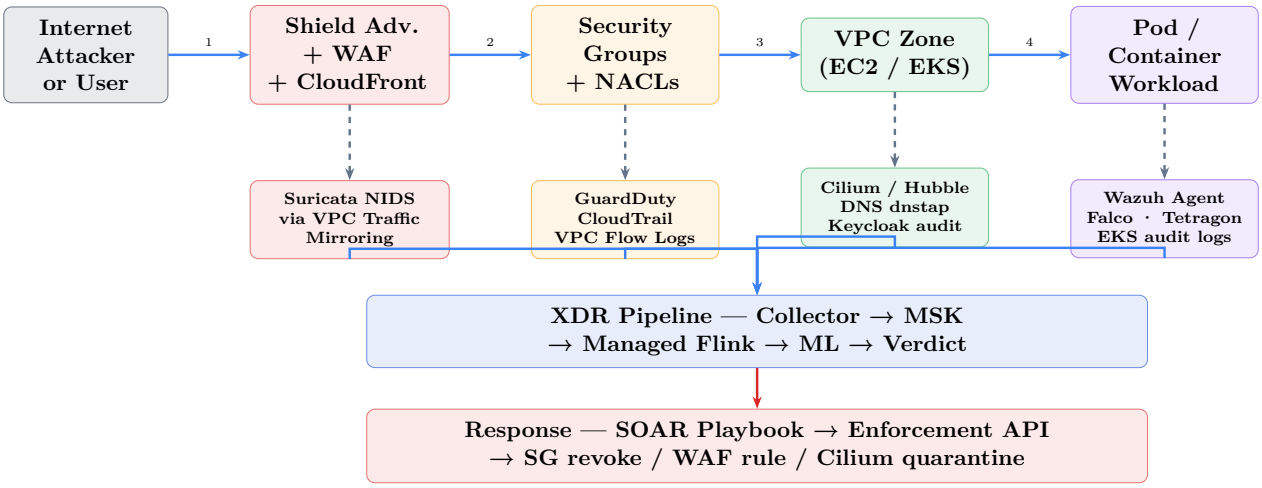


Figure 2: End-to-end AWS traffic flow. Each network hop is covered by a different sensor layer. All sensor output converges at the XDR pipeline, which drives automated response.

Detection Layers — The Seven Sensors

The detection architecture deploys **seven independent sensor layers**. Each layer operates simultaneously. A threat that evades one will likely be caught by another.

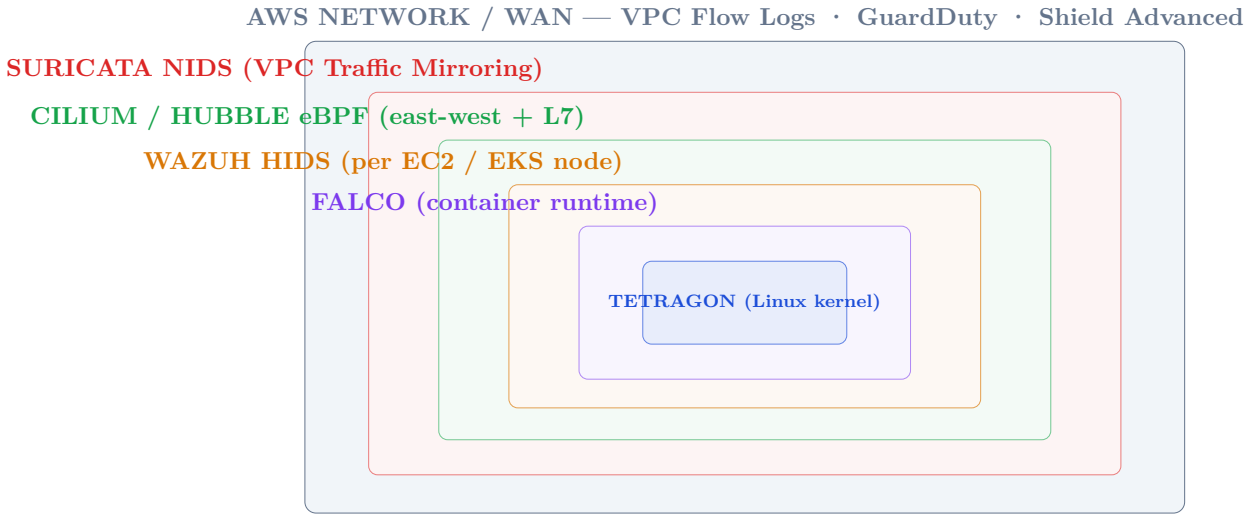


Figure 3: Sensor layers, from widest coverage (outer) to deepest visibility (inner). Each inner layer provides more granular visibility but only within its scope.

Layer 1 — Tetragon (Linux Kernel, $< 1\,\mu$s)

Tetragon — The In-Kernel Enforcer

What it is: An eBPF-based security enforcer embedded inside the Linux kernel on every EKS node. Using eBPF, Tetragon attaches tiny programs directly to kernel hooks that run *before* any user-space code can observe or react.
What it watches: Every `execve()` (process launch), file open, network `connect()`, privilege escalation attempt (`setuid`, capability changes).

Autonomous action: If an event matches a TracingPolicy (e.g. `nc` or `nmap` executes inside a pod), Tetragon sends `SIGKILL` in under 1 microsecond. The malicious process is dead before any payload is written to disk or any packet leaves the container.
Key point: Tetragon is *Tier-1 autonomous* — it operates with zero dependency on SOAR, Kafka, or the ML pipeline. It continues to protect even if the rest of the XDR stack is offline.

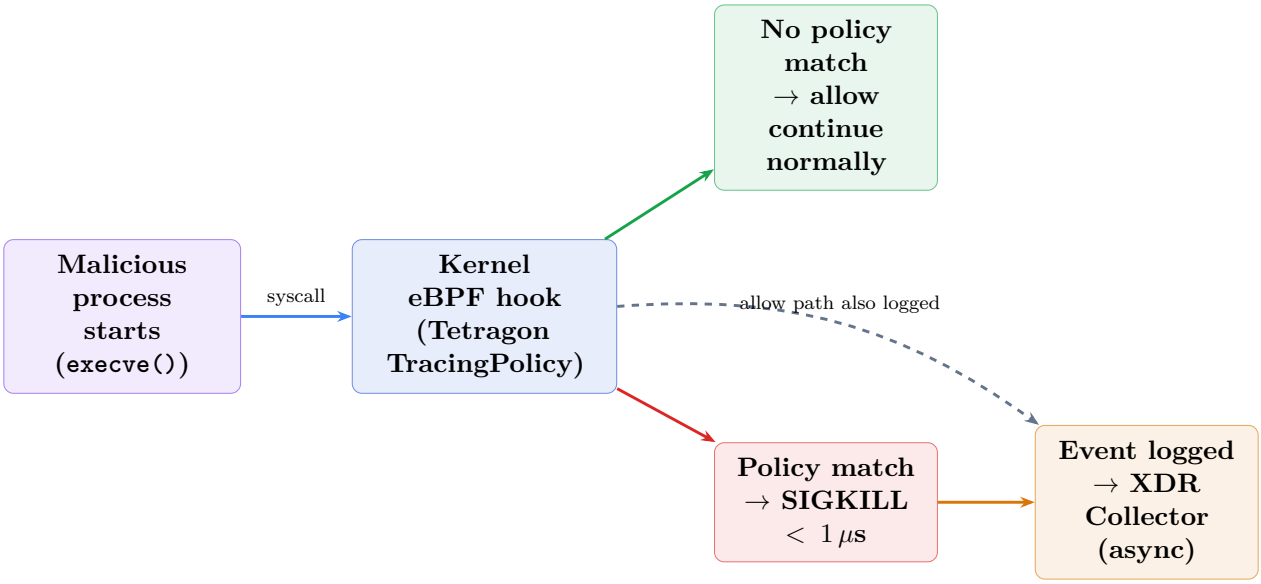


Figure 4: Tetragon autonomous kill path. The decision and kill happen entirely in kernel space, before any user-space security tool can even receive the event.

Layer 2 — Falco (Container Runtime, milliseconds)

Falco — Container Behaviour Monitor

What it is: A cloud-native runtime security tool. Falco knows what a "normal" container looks like from its image definition, and raises an alert the moment behaviour deviates.
What it detects on EKS:

- A shell spawning inside a container that should not have one (*Terminal shell in container*)
- A container writing to `/etc/` or `/bin/`
- Sensitive file reads: `/etc/shadow`, `/root/.ssh/`
- Network reconnaissance tools (`nmap`, `masscan`) executing inside a pod
- EKS API server calls from unexpected processes

Output: Fires an alert within milliseconds → forwarded to the XDR Collector.

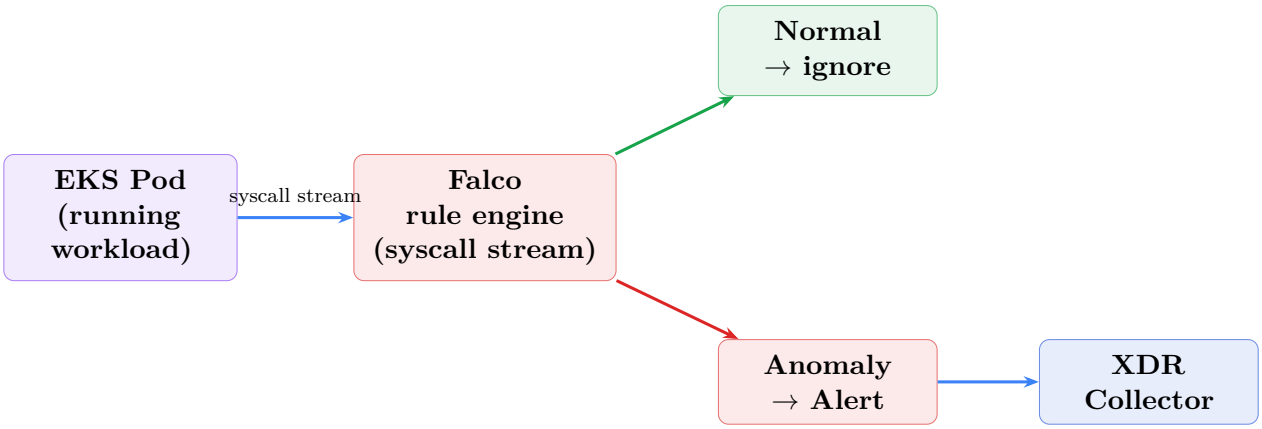


Figure 5: Falco runtime detection flow on EKS. Every container syscall is evaluated against rules; deviations produce alerts that enter the XDR pipeline.

Layer 3 — Wazuh HIDS (Host, 50 ms — seconds)

Wazuh — Host Intrusion Detection System

What it is: A HIDS with an agent installed on every EC2 instance and EKS node, reporting to a central Wazuh Manager deployed on EKS.

What it watches:

- **File Integrity Monitoring:** "Did someone change `/etc/passwd`?"
- **Log analysis:** "50 failed SSH logins in 10 seconds = brute force"
- **Process auditing:** "Is something running as root that should not be?"
- **3,000+ built-in detection rules**
- **SSM integration:** Wazuh active response can also trigger AWS SSM Run Command for broader remediation

Autonomous Tier-1 response: Wazuh automatically issues a `firewall-drop` command within **50 ms** — blocking the attacking IP at the host firewall without waiting for SOAR or analyst approval.

Wazuh: Standalone vs. XDR Source

This is a critical architectural point. Wazuh sits in the **Control Plane**, not inside the XDR pipeline. It is a peer-level system with its own complete stack, and *also* feeds processed alerts into XDR.

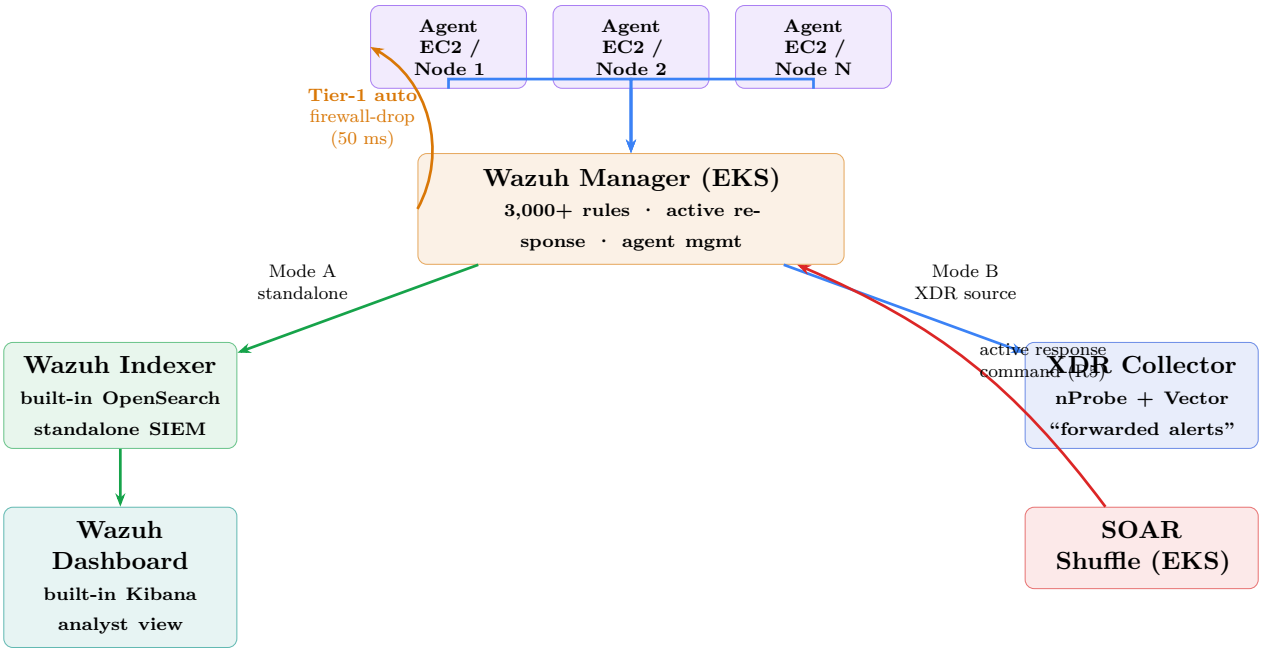


Figure 6: Wazuh dual-mode operation. Mode A (standalone loop, always active) provides immediate host-level SIEM and autonomous response. Mode B adds multi-source XDR correlation. Both run simultaneously.

What Wazuh Covers Alone vs. What Needs Full XDR

Use Case	Wazuh standalone	Needs full XDR
File integrity monitoring	**Yes**	No
Log analysis + rule-based alerts	**Yes**	No
Brute-force detect + auto-block	**Yes (Tier 1, 50 ms)**	No
Vulnerability scanning	**Yes (built-in)**	No
Compliance (PCI-DSS, GDPR, CIS)	**Yes**	No
SIEM storage + dashboards	**Yes**	No
Network-layer attack detection	**No**	Suricata NIDS
Container runtime detection	**No**	Falco / Tetragon
East-west container flow visibility	**No**	Cilium / Hubble
DNS tunnel detection	**No**	DNS monitor + ML
Lateral movement (graph)	**No**	GNN in XDR ML
Multi-source ML correlation	**No**	Full XDR pipeline

Layer 4 — Cilium + Hubble (East-West, sub-second)

Cilium — The EKS CNI with Security Superpowers

What it is: Cilium is the CNI (Container Network Interface) for the EKS cluster — the system that gives pods their IP addresses and controls their network access. It uses eBPF in the kernel for both policy enforcement and observability.

What it does:

- **Network policy:** "This pod may only talk to the database pod on port 5432, nothing else" — enforced in the kernel, not by a software proxy
- **Hubble:** Logs every single flow between pods (who talked to who, what port, was it allowed or denied). This data is fed into the XDR Collector as `det-ebpf` telemetry
- **Quarantine labels:** When SOAR orders a pod quarantined, Cilium applies a label that makes every other pod refuse to communicate with it — instant network isolation without `kubectl delete`
- **WireGuard:** Encrypts all inter-node pod traffic transparently — mTLS for free
- **L7 visibility:** Hubble can log HTTP method + URL, gRPC method, and Kafka topic for any pod without a sidecar

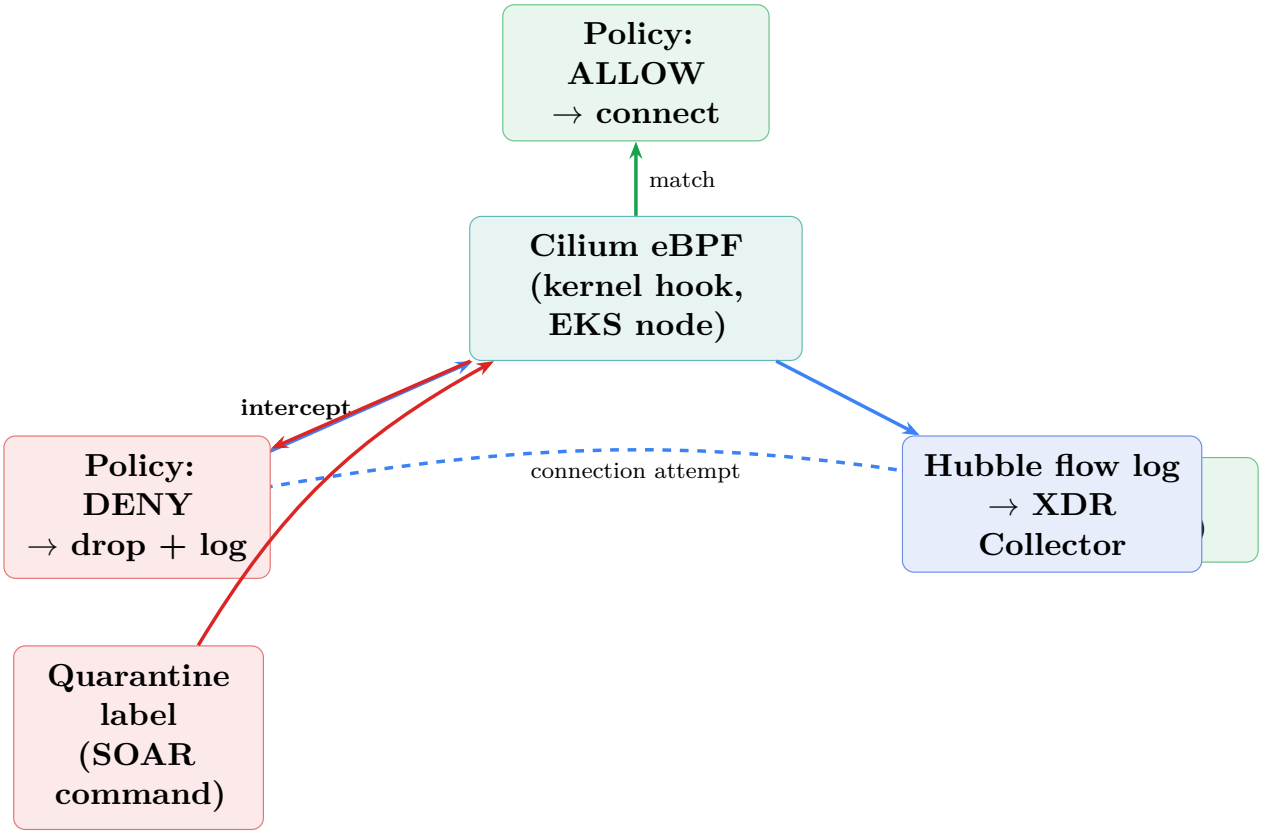


Figure 7: Cilium policy enforcement and Hubble observability on EKS. The quarantine label path shows how SOAR isolates a compromised pod without stopping it — preserving forensic evidence.

Layer 5 — Suricata NIDS (Network, near real-time)

Suricata — The Network Intrusion Detection System

What it is: A NIDS that inspects raw network packets. In AWS, it receives a copy of all VPC traffic via **VPC Traffic Mirroring** and runs on a dedicated Suricata EC2 instance.

What it detects:

- Known malware C2 traffic patterns and signatures
- Port scan patterns (many connections to different ports)
- SYN flood patterns (DDoS precursor)
- Exploit payloads in unencrypted protocols (HTTP, FTP, DNS)

AWS-Specific Limitation

VPC Traffic Mirroring copies traffic from ENIs (Elastic Network Interfaces) to the Suricata EC2 instance. This has two constraints:

- **Cost:** Mirrored traffic is billed per GB — high-traffic environments incur significant charges
- **Bandwidth:** There is a cap on mirroring throughput per ENI, which means Suricata may not see 100% of traffic under heavy load

For network-level real-time detection, **GuardDuty** complements Suricata precisely because it does not depend on traffic mirroring.

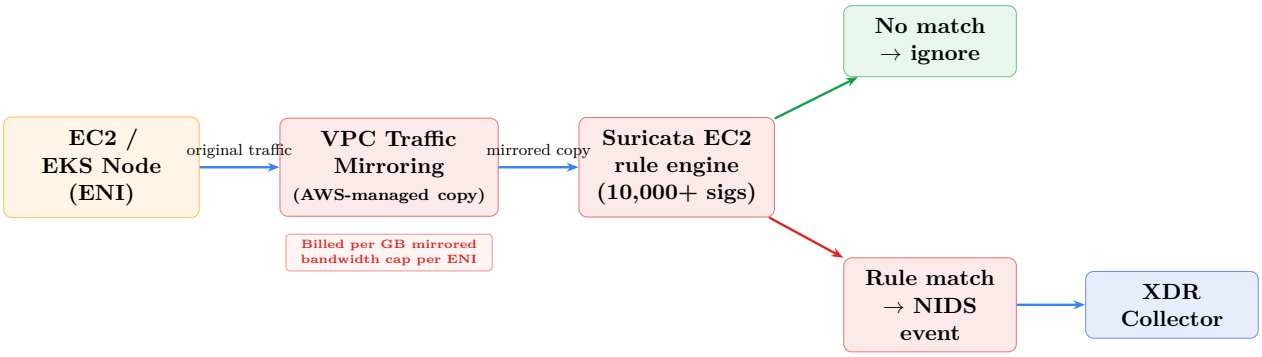


Figure 8: Suricata NIDS on AWS. Traffic is mirrored at the ENI level to a dedicated Suricata EC2 instance. The original traffic is unaffected.

Layer 6 — AWS-Native Security Services (GuardDuty, CloudTrail, etc.)

AWS-Native Detection — No Agents Required

AWS provides a set of fully managed security services that require zero deployment or maintenance. They operate at the AWS infrastructure layer and are therefore immune to agent-based tampering.

- **GuardDuty:** ML-based threat detection on VPC Flow Logs, CloudTrail, and DNS logs. Detects crypto mining, C2 beaconing, credential exfiltration, impossible

travel. Response latency 1–5 min. Can auto-block via Lambda.

- **CloudTrail:** Records every single AWS API call (who, what, when, from where). Essential for detecting IAM abuse, privilege escalation, and resource exfiltration.
- **Security Hub:** Aggregates findings from GuardDuty, Inspector, Macie, and third-party tools into a single dashboard.
- **Inspector:** Continuous vulnerability scanning of EC2 and container images.
- **Macie:** Detects sensitive data (PII, credentials) in S3 buckets.

VPC Flow Log Delay — Critical AWS Limitation

VPC Flow Logs are the primary source of network flow telemetry in AWS. However, they have a **1–10 minute publication delay** into CloudWatch Logs. This means:

- Flow logs **cannot** be used for real-time DDoS detection or lateral movement detection
- Real-time network coverage relies on **Suricata** (NIDS via mirroring) and **GuardDuty**
- If the Suricata EC2 instance is unavailable, there is a multi-minute network blind spot

Layer 7 — DNS Monitoring + Identity / Auth Logs

DNS and Auth — The Phonebook and Badge Scanner

Route 53 Query Logging: Every DNS query made by any EC2 or EKS workload is logged into CloudWatch Logs and forwarded to Kafka. This catches DNS tunnelling (data encoded in query names), DGA domains (random-looking names generated by malware), and lookups to known-bad domains from threat intelligence feeds.

CloudTrail + Cognito/Keycloak audit: Every AWS API call and every login event is logged. This catches:

- Brute-force / credential stuffing (50 failed logins in 30 s)
- Impossible travel (login from two locations minutes apart)
- AWS API key abuse (API calls from unexpected regions or at unusual times)
- EKS API server audit logs (who ran `kubectl exec` into which pod)

The XDR Pipeline — Connecting All the Dots

The Control Room

XDR is the control room that receives feeds from all sensor layers, correlates them with AI and ML, and produces a single actionable verdict. One Falco alert alone might be noise; combined with a Hubble denied-flow to another zone and a GuardDuty finding — it is a confirmed lateral movement attempt.

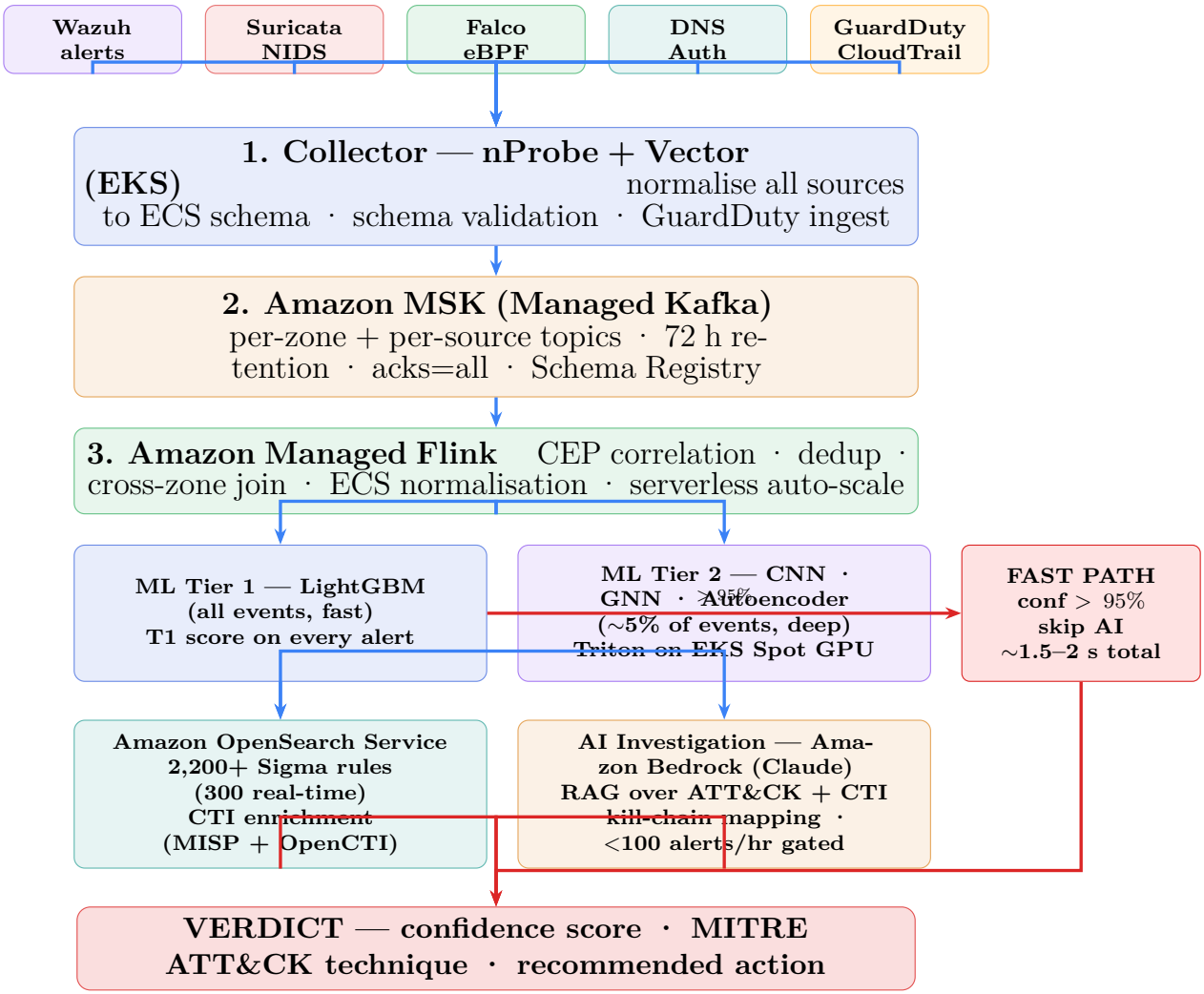


Figure 9: Full XDR pipeline on AWS. The Fast Path (red dashed route) bypasses AI investigation for high-confidence (> 95%) events, reducing total detection-to-block time to approximately 1.5–2 seconds.

The Message Queue — Why Amazon MSK Matters

Amazon MSK as Shock Absorber

MSK (Managed Streaming for Apache Kafka) acts as a durable buffer between the Collector and the Flink processor. During a DDoS attack or any event spike, the Collector pushes millions of events per second into Kafka topics. Flink reads at its own sustainable rate — nothing is lost. Without the queue, a traffic spike would crash the stream processor at exactly the moment you need it most.

Key settings: `acks=all` (no data loss on broker failure) · 72-hour retention · Schema Registry (type-safe events) · per-zone and per-source topics (blast radius isolation).

ML Tiers and the Fast Path

Tier	Model	What it checks	When triggered
T1	LightGBM (gradient boost)	All events — fast scalar features	100% of events
T2	CNN + MLP	Sequence patterns, time-series	∼5% (T1 flagged as suspicious)
T2	GNN (graph net)	Lateral movement — connection graph anomalies	∼5%
T2	Autoencoder	Anomaly detection (unseen patterns)	∼5%
T2	DGA / DNS tunnel	DNS query statistical analysis	DNS events
AI	Amazon Bedrock (Claude)	Kill chain mapping, MITRE ATT&CK, narrative	∼1%

The Fast Path

If T1 or T2 returns confidence > 95%, the event is sent directly to verdict — bypassing the AI investigation step (which takes 5–60 seconds). This path handles well-known, high-confidence attacks such as DNS tunnelling or brute-force, achieving a full detection-to-block cycle of **approximately 1.5–2 seconds**.

SIEM, SOAR, and XDR — What Each Platform Does

These three terms are often confused. The table below defines their boundaries.

	SIEM (OpenSearch)	**SOAR** (Shuffle)	**XDR** (full pipeline)
Role	Store, search, and rule-match security events	Execute automated response playbooks	Correlate events across all sources with ML
Analogy	Filing cabinet + alarm rulebook	Manager who carries out orders	Control room combining all camera feeds
Input	Raw + normalised events	XDR verdict	All detection sources
Output	Sigma alerts, stored logs	Network blocks, quarantines, tickets	Verdict + confidence + ATT&CK mapping
Autonomous?	Yes (Sigma rules fire automatically)	Yes (playbooks auto-execute)	Yes (ML inference)
Needs humans for?	Complex investigations, rule tuning	Writing playbooks, high-impact approvals	Model retraining, FP/TP labelling

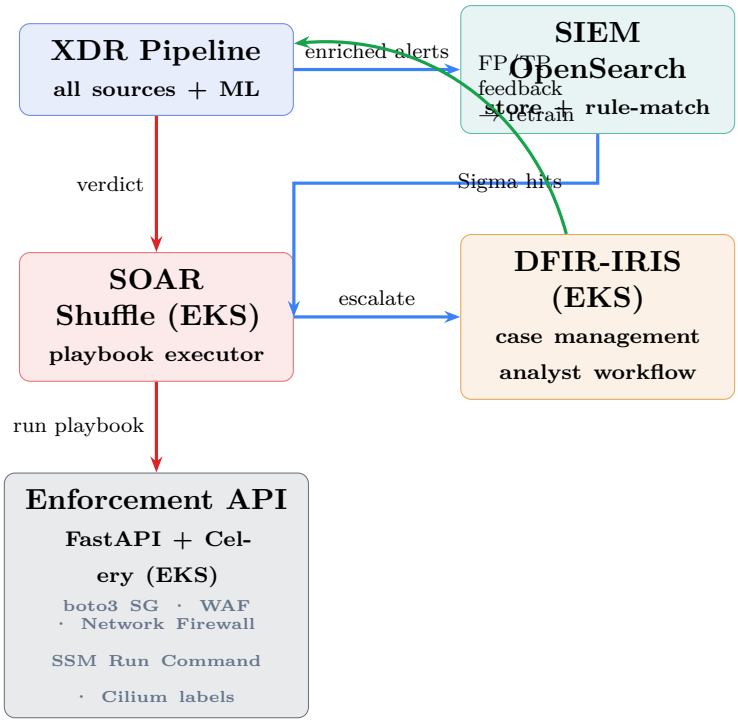


Figure 10: Relationship between XDR, SIEM, SOAR, and Case Management on AWS. SOAR is the executor; SIEM is the store and rule library; XDR is the correlating brain; DFIR-IRIS closes the feedback loop to improve ML accuracy over time.

SDN in AWS — Why There Is No Controller

AWS VPC *is* the SDN

Traditional SDN (Software-Defined Networking) uses a separate controller (OpenFlow, gNMI) to program physical or virtual switches. In AWS, there are no switches to program — the **AWS VPC control plane itself** is the programmable network layer. All network enforcement happens via AWS API calls using `boto3`.

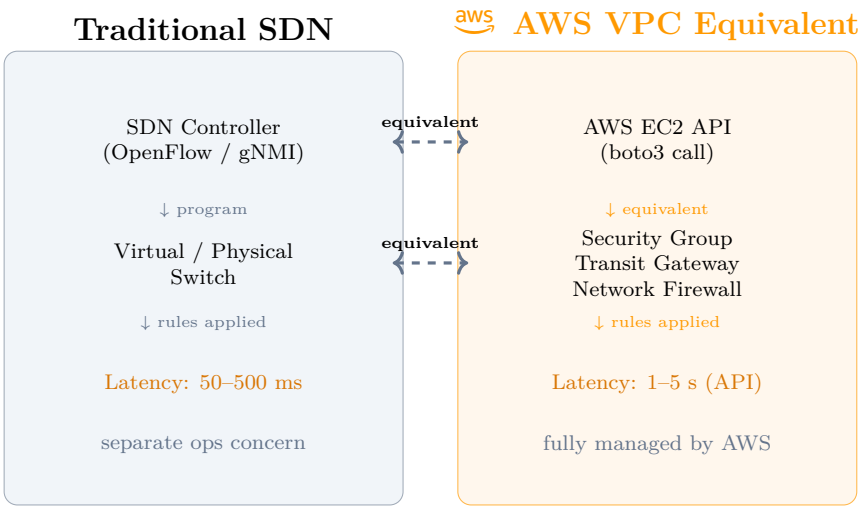


Figure 11: Traditional SDN vs. AWS VPC. In AWS, there is no separate controller to manage. The API call *is* the SDN control message. This simplifies operations but introduces 1–5 s API latency vs. sub-100 ms for gNMI on physical switches.

Enforcement API — The AWS Response Worker

The **Enforcement API** is a FastAPI + Celery service running on EKS with four boto3 workers:

- **SG worker:** `boto3 ec2.authorize_security_group_ingress / revoke` — blocks a source IP in 1–5 s
- **WAF worker:** Adds or removes rate rules and IP block lists on AWS WAF
- **Network Firewall worker:** Pushes stateful rules to AWS Network Firewall (managed NGFW)
- **SSM worker:** Executes run commands on EC2 instances via SSM (e.g. `iptables` rules, process kill, file collection)

All operations are **idempotent** (safe to run twice) and **TTL-based** (rules auto-expire after a configurable duration to prevent stale blocks).

The Tiered Defense Model

The Immune System

The human immune system has an innate response (fast, automatic, kills anything foreign) and an adaptive response (slower, learned, more targeted). This architecture mirrors that model: Tier 1 fires automatically in under 50 ms; Tier 2 runs targeted SOAR playbooks in 1–5 seconds. Tier 0 is the preventive layer — it never needs to detect anything because it simply does not allow it.

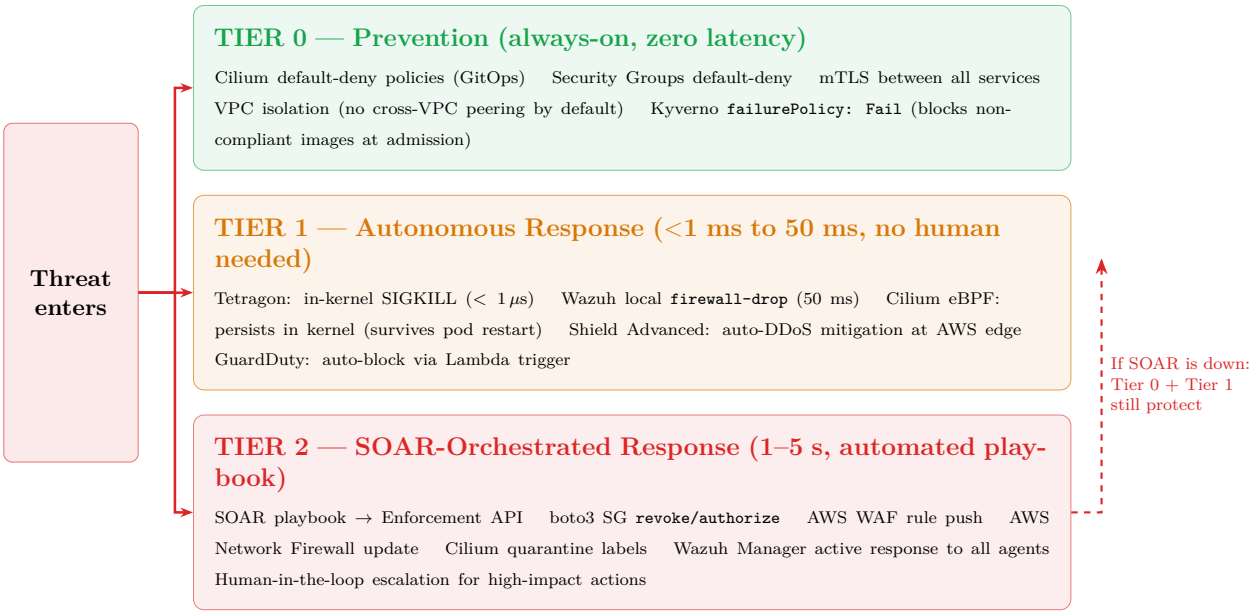


Figure 12: Three-tier defense model on AWS. Each tier is independent. Tier 0 is the most reliable because it requires no detection — it simply does not permit the action. Tier 1 is pre-programmed autonomous response. Tier 2 runs SOAR playbooks with optional human approval for high-impact actions.

Attack Scenario Walkthroughs

Scenario 1 — DNS Tunnel (C2 over DNS)

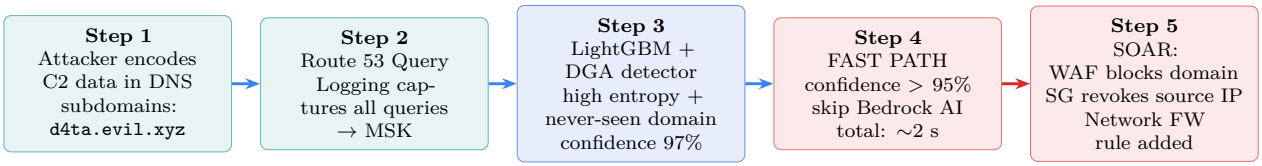


Figure 13: DNS tunnel detection and response on AWS. Route 53 Query Logging is the key data source; ML fast-paths the verdict in ∼2 seconds.

Scenario 2 — Lateral Movement (Zone B → Zone A)

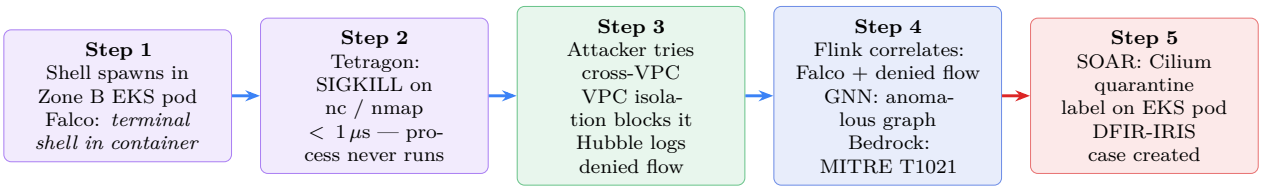


Figure 14: Lateral movement scenario on AWS EKS. Tetragon kills tools; VPC isolation blocks cross-zone traffic; XDR correlates the multi-source evidence; SOAR quarantines the compromised pod.

Scenario 3 — Brute Force + Credential Abuse

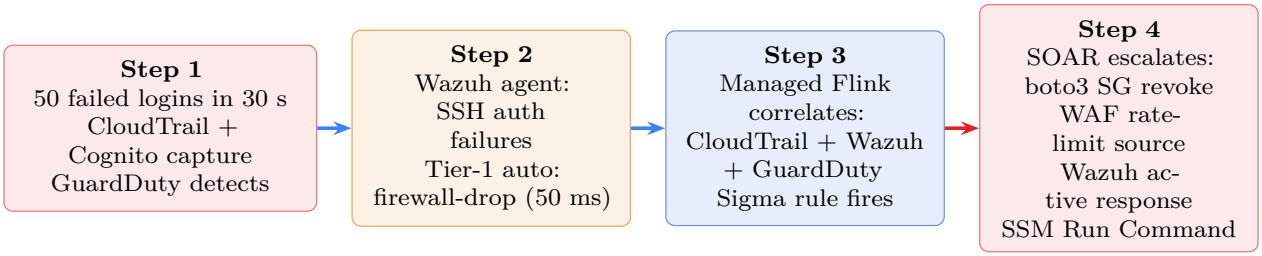


Figure 15: Brute force on AWS. Wazuh's Tier-1 response fires in 50 ms — before SOAR even receives the XDR verdict. The XDR pipeline then escalates with a broader coordinated block.

Scenario 4 — DDoS Attack

AWS Flow Log Delay in a DDoS

VPC Flow Logs have a 1–10 minute delay. In a DDoS, this means flow-log-based detection arrives *after* the attack has already been impacting the environment. Shield Advanced and Suricata are the real-time defences. Flow logs provide post-event forensics only.

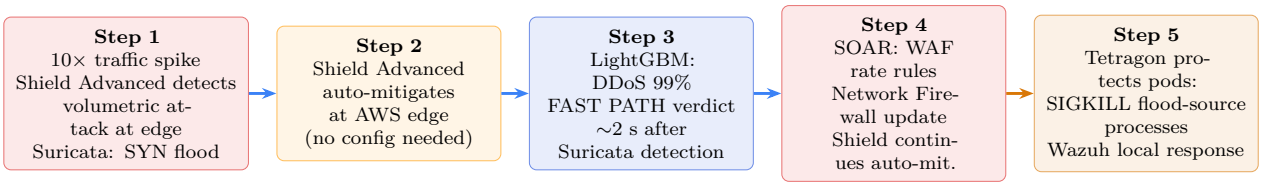


Figure 16: DDoS on AWS. Shield Advanced is the primary defence and operates autonomously. XDR adds WAF and Network Firewall rules while Tetragon protects individual EKS pods.

Scenario 5 — Container Compromise

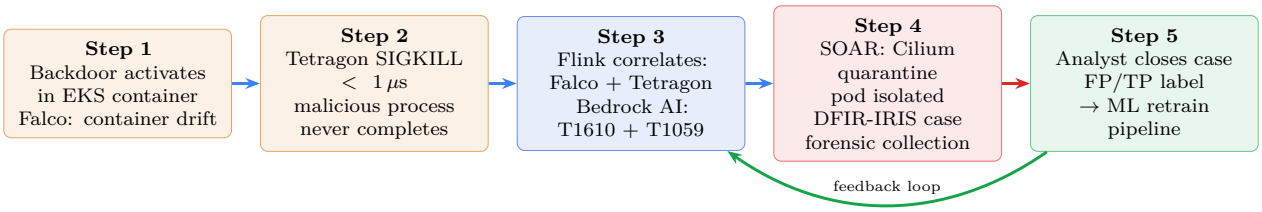


Figure 17: Container compromise on AWS EKS. The feedback loop in Step 5 is what makes the ML model improve over time — analyst-confirmed detections become training data.

AWS Tool Reference

Tool	Type	AWS Deployment	Role
Wazuh Manager	HIDS Manager	EKS (Kubernetes)	Central agent manager, rule engine, active response dispatcher
Wazuh Agent	HIDS Agent	Every EC2 / EKS node	Host-level monitoring, FIM, log analysis, Tier-1 auto response
Suricata	NIDS	Dedicated EC2	Packet-level network intrusion detection via VPC Traffic Mirroring
Cilium	CNI + eBPF	EKS (DaemonSet)	Container network policy, quarantine labels, WireGuard, L7 visibility
Hubble	Observability	EKS (sidecar-free)	Flow logs for all pod-to-pod communication fed to XDR
Tetragon	eBPF enforcer	EKS (DaemonSet)	In-kernel autonomous SIGKILL, Tier-1 autonomous response
Falco	Runtime sec.	EKS (DaemonSet)	Container behaviour monitoring, drift detection
GuardDuty	AWS-native	AWS managed	ML threat detection on VPC Flow Logs, CloudTrail, DNS logs
Security Hub	AWS-native	AWS managed	Aggregates findings from GuardDuty, Inspector, Macie
CloudTrail	AWS-native	AWS managed	API call logging for all AWS actions
Route 53 Logs	DNS logging	AWS managed	DNS query logging for tunnel and DGA detection
Shield Advanced	DDoS protect.	AWS managed	Automated DDoS mitigation at AWS edge
AWS WAF	Web FW	CloudFront / ALB	Rate rules, IP blocks, SQLi/XSS protection
nProbe + Vector	Collector	EKS	Telemetry normalisation to ECS schema, GuardDuty ingest
Amazon MSK	Message queue	AWS managed Kafka	Durable event buffer, 72 h retention, Schema Registry

Tool	Type	AWS Deployment	Role
Managed Flink	Stream proc.	AWS managed	CEP correlation, dedup, cross-zone joins, serverless scaling
Triton on EKS	ML inference	EKS Spot GPU	Serves LightGBM, CNN, GNN, Autoencoder, DGA models
OpenSearch Svc	SIEM	AWS managed	2,200+ Sigma rules, hot/warm/cold storage, analyst search
MISP + OpenCTI	CTI	EKS	Threat intelligence feeds, STIX 2.1, ATT&CK mapping
Amazon Bedrock	LLM AI	AWS managed (Claude)	AI investigation, kill-chain mapping, ATT&CK narrative
Shuffle SOAR	SOAR	EKS	Playbook automation, graduated response, confidence thresholds
DFIR-IRIS	Case mgmt	EKS	Incident lifecycle, FP/TP feedback to ML retraining pipeline
Trivy	Supply chain	EKS admission webhook	Container image vulnerability scanning at build + deploy
Kyverno	Policy engine	EKS admission controller	Enforces image signing, resource policies; `failurePolicy: Fail`
Sigstore	Supply chain	CI/CD + EKS	Cryptographic container image signing verification

Gaps in the AWS Deployment

Critical Gaps

1. **VPC Flow Log delay (1–10 min)** — Real-time network detection relies entirely on Suricata (VPC mirroring) and GuardDuty. If Suricata is unavailable, there is a multi-minute network blind spot. Flow logs cannot substitute for real-time NIDS.
2. **No encrypted traffic inspection** — Suricata cannot inspect HTTPS/TLS payloads without a TLS termination proxy in the path. eBPF (Hubble) sees

connection metadata but not decrypted application-layer content. The majority of modern attacks use HTTPS.

3. **No email / phishing detection** — Phishing is the #1 initial access vector (MITRE T1566) and there is zero inbound email security layer in the architecture. An account compromised via phishing would only be caught at the post-login stage (CloudTrail / GuardDuty brute-force).
4. **VPC Traffic Mirroring bandwidth cap** — Suricata receives a mirrored copy of ENI traffic. Under heavy load, mirroring throughput is capped per ENI, which means Suricata may miss packets during high-traffic periods.

Significant Gaps

1. **ML retraining pipeline not shown** — The DFIR-IRIS feedback loop to ML retraining is drawn as an arrow, but the actual model versioning, validation, canary deployment, and A/B testing pipeline is absent from the design.
2. **No deception layer** — No honeypots, honeytokens, canary DNS names, canary AWS IAM credentials, or S3 canary objects. Slow, low-and-slow attackers will not trigger any threshold-based detection.
3. **No UEBA** — No dedicated User and Entity Behaviour Analytics for long-term baseline deviation (insider threats, slow account takeovers that stay below per-minute thresholds).
4. **Supply chain response gap** — Trivy + Kyverno block non-compliant images at admission. However, there is no documented SOAR playbook for a supply chain compromise that has *already been deployed* (e.g. a compromised base image that passed all checks at build time).
5. **No explicit EDR** — Wazuh provides HIDS capability; Tetragon and Falco provide runtime protection. However, there is no full EDR-class capability: memory forensics, process injection detection, kernel rootkit scanning, or live response console.
6. **GuardDuty latency (1–5 min)** — GuardDuty is not real-time. It analyses VPC Flow Logs and CloudTrail in near-real-time windows but findings typically arrive 1–5 minutes after the event. This is acceptable for most scenarios but not for fast-moving lateral movement.
7. **Cost sensitivity of VPC Traffic Mirroring** — At high traffic volumes, mirrored traffic costs can escalate significantly. There is no documented cost cap or automatic mirroring scope reduction policy.

AWS Deployment Metrics Summary

Metric	Value / Notes	Bottleneck / Concern
Detection latency (network)	Flow logs: 1–10 min delay; NIDS: near real-time; GuardDuty: 1–5 min	**Flow log delay is the main gap**
Detection latency (host)	Sub-second (Wazuh + Tetragon + Falco)	**No concern**
Response latency (autonomous)	$< 1\,\mu$s (Tetragon), 50 ms (Wazuh)	**Best-in-class**
Response latency (API)	1–5 s (boto3 SG / WAF / NW Firewall)	**Acceptable for most threats**
Response latency (Shield)	Automatic at AWS edge, <seconds	**No concern**
ML pipeline latency	∼1.5–2 s (fast path), 5–60 s (with Bedrock AI)	**Fast path handles high-vol**
OpenSearch rule capacity	2,200+ Sigma rules; 300 real-time	**Rule tuning cadence needed**
Monthly cost	$15,000–17,000	**Managed services reduce ops burden**
Scalability	Elastic auto-scaling (EKS + MSK + Flink + OpenSearch)	**No capacity planning needed**
DDoS protection	Shield Advanced (AWS-managed, always-on)	**Best available on AWS**
Scenario ratings	DNS tunnel: 4/5 · Lateral move: 4/5 · Brute force: 5/5 · DDoS: 5/5 · Container: 5/5	**Lateral movement limited without GNN running at full capacity**